

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INFORMATICA



Corso Penetration Testing and Ethical Hacking

***PrioritAIze: Contextual Vulnerability Risk Scoring
via LLM Platform***

Professore:

Arcangelo Castiglione

Candidato:

Luigi Miranda
Matricola: 0522501934

ANNO ACCADEMICO: 2025/2026

1. Introduzione

I moderni team di sicurezza affrontano un volume crescente di vulnerabilità segnalate da scanner automatici, penetration test, bug bounty program e feed di threat intelligence. La metrica standard per la prioritizzazione — il Common Vulnerability Scoring System (CVSS) — valuta la gravità **tecnica** di una vulnerabilità su scala 0–10, ma ignora completamente il contesto aziendale dell’asset colpito: la sua criticità operativa, l’esposizione di rete, i framework normativi applicabili e l’impatto finanziario di un eventuale breach.

Questo report descrive **PrioritAIze**, un framework che combina:

1. **Large Language Model (LLM) reasoning** per analizzare qualitativamente il rischio nel contesto specifico dell’asset;
2. **Business Impact Modeling** per quantificare l’impatto finanziario in Euro (costi di downtime, sanzioni normative, danno reputazionale);
3. **Dynamic Risk Scoring** che produce un punteggio 0–10 pesato contestualmente, superando la staticità del CVSS.

Il risultato è una lista di vulnerabilità prioritizzata non per gravità tecnica, ma per **impatto reale sul business**, consentendo ai team di security di allocare le risorse dove generano il massimo valore di riduzione del rischio.

1.0.1. Obiettivi del progetto

- Sviluppare un sistema containerizzato, riproducibile, con architettura modulare a cinque componenti;
- Integrare un LLM (qualsiasi modello OpenAI-compatibile) per il ragionamento contestuale sul rischio;
- Implementare un modello di impatto finanziario deterministico che simuli downtime, multe normative e danno reputazionale;
- Produrre una dashboard web interattiva con trasparenza totale sulle formule di calcolo.

1.0.2. Autorizzazione e scope

Il progetto opera esclusivamente su dati mock (seed data in `assets.yaml` e `cost_model.yaml`). Non è stato effettuato alcun test su sistemi, reti o asset reali. L’architettura è progettata per l’integrazione diretta con fonti live (CMDB, cloud inventory API, criticality tagging systems) in un ambiente produttivo autorizzato.

2. Stato dell'arte

2.0.1. Sistemi di scoring tradizionali

Il **Common Vulnerability Scoring System (CVSS)**, mantenuto dal FIRST (Forum of Incident Response and Security Teams), è lo standard de facto per la valutazione della gravità delle vulnerabilità. Nella versione 3.1, il punteggio è calcolato a partire da tre gruppi di metriche:

- **Base Score:** caratteristiche intrinseche della vulnerabilità (vettore d'attacco, complessità, privilegi richiesti, impatto su confidenzialità / integrità / disponibilità);
- **Temporal Score:** fattori che cambiano nel tempo (maturità dell'exploit, disponibilità di patch);
- **Environmental Score:** adattamento all'ambiente specifico (requisiti di sicurezza dell'organizzazione).

Il CVSS Environmental Score consente un adattamento limitato — principalmente tramite la modifica dei requisiti di confidenzialità, integrità e disponibilità (CIA) — ma non incorpora dati finanziari, normativi o di business criticality specifici dell'asset. Nella pratica, la maggior parte delle organizzazioni utilizza solo il Base Score per la prioritizzazione, perdendo ogni differenziazione contestuale [1].

2.0.2. Approcci basati su rischio

Negli ultimi anni sono emersi framework di **risk-based vulnerability management (RBVM)** che arricchiscono il CVSS con dati di contesto:

- **Tenable Vulnerability Priority Rating (VPR):** combina CVSS con dati di threat intelligence e maturity degli exploit tramite modelli di machine learning [3].
- **Kenna.VM (Cisco):** utilizza algoritmi predittivi per stimare la probabilità di exploit effettivo.
- **Qualys TruRisk:** aggrega dati di asset criticality, configurazioni di sicurezza e threat intelligence.

Questi sistemi sono efficaci ma sono **proprietary, closed-source** e offrono all'utente un controllo limitato sulla logica di scoring. Inoltre, nessuno di essi utilizza LLM per il ragionamento qualitativo contestuale [2].

2.0.3. LLM nella cybersecurity

L'applicazione di Large Language Models alla cybersecurity è un campo in rapida evoluzione. Lavori recenti hanno esplorato:

- **Vulnerability detection:** modelli come GPT-4 e modelli specializzati (es. SecureBERT, CyBERT) per identificare vulnerabilità nel codice sorgente;
- **Threat report summarization:** sintesi automatica di report di intelligence;
- **Automated penetration testing:** agenti LLM-based per la generazione di payload e il ragionamento su catene di exploit.

Il presente progetto si colloca nell'intersezione tra LLM reasoning e risk-based vulnerability management, un'area ancora poco esplorata nella letteratura accademica e commerciale. L'innovazione principale risiede nell'approccio **ibrido**: l'LLM fornisce il ragionamento qualitativo (perché una vulnerabilità è più o meno grave in un dato contesto), mentre formule deterministiche ancorate a parametri configurabili calcolano l'impatto finanziario quantitativo.

3. Tecnologie utilizzate

Tecnologia	Versione / Tipo	Ruolo nel sistema
Python	3.12	Linguaggio principale — ecosistema ricco per AI/ML, API, data processing
FastAPI	0.115+	Web framework asincrono — validazione automatica, documentazione OpenAPI integrata
Jinja2	3.x	Template engine per server-side rendering delle pagine web
Bootstrap 5	5.3	CSS framework per UI responsive, supporto nativo dark mode tramite data-bs-theme
Chart.js	4.x	Libreria JavaScript per grafici dashboard (istogramma, doughnut, pie)
PostgreSQL	16-alpine	Database relazionale per persistenza delle valutazioni e audit log
psycopg2	2.9	Driver PostgreSQL — accesso raw SQL con RealDictCursor, nessun ORM
OpenAI SDK	1.x	Client compatibile con qualsiasi endpoint OpenAI-compatibile (GPT-4, Claude, Llama, DeepSeek, Ollama)
httpx	0.27+	Client HTTP asincrono per chiamate alla NVD API v2
uv	0.6+	Package manager veloce — sostituisce pip + venv
Docker Compose	v2+	Orchestrazione container — app + PostgreSQL in ambienti riproducibili
Ruff	0.9+	Linter e formatter — sostituisce flake8 + black + isort in un unico tool

Tabella 1: Stack tecnologico del progetto PrioritAIze.

3.0.1. Motivazione delle scelte

FastAPI è stato scelto per il supporto nativo all'asincronia (essenziale per le chiamate concorrenti a LLM e NVD), la validazione automatica dei dati (Pydantic) e la generazione automatica della documentazione OpenAPI.

PostgreSQL con raw SQL: il progetto richiede esplicitamente l'uso di SQL diretto senza ORM. L'uso di RealDictCursor garantisce un'esperienza ergonomica (dizionari Python) simile a un ORM senza il layer di astrazione.

Server-side rendering con Jinja2: la scelta di evitare framework SPA (React, Vue) riduce la complessità del build toolchain, garantisce caricamenti più veloci (HTML completo al primo render) e risulta più adatta a un tool interno/PoC accademico. L'interattività è delegata a Chart.js e vanilla JavaScript per le parti dinamiche della dashboard.

Compatibilità multi-LLM: l'uso dell'OpenAI SDK con `base_url` configurabile consente di puntare a qualsiasi endpoint compatibile (OpenAI, Anthropic Claude via proxy, Llama locale via Ollama, DeepSeek, vLLM), rendendo il sistema indipendente da un singolo fornitore.

4. Metodologia e architettura

4.0.1. Architettura del sistema

Il sistema è composto da cinque moduli interconnessi, orchestrati da una **pipeline di valutazione** che processa ciascuna vulnerabilità in sei fasi sequenziali. L'architettura è containerizzata (Docker Compose) e ogni componente è indipendente e sostituibile.

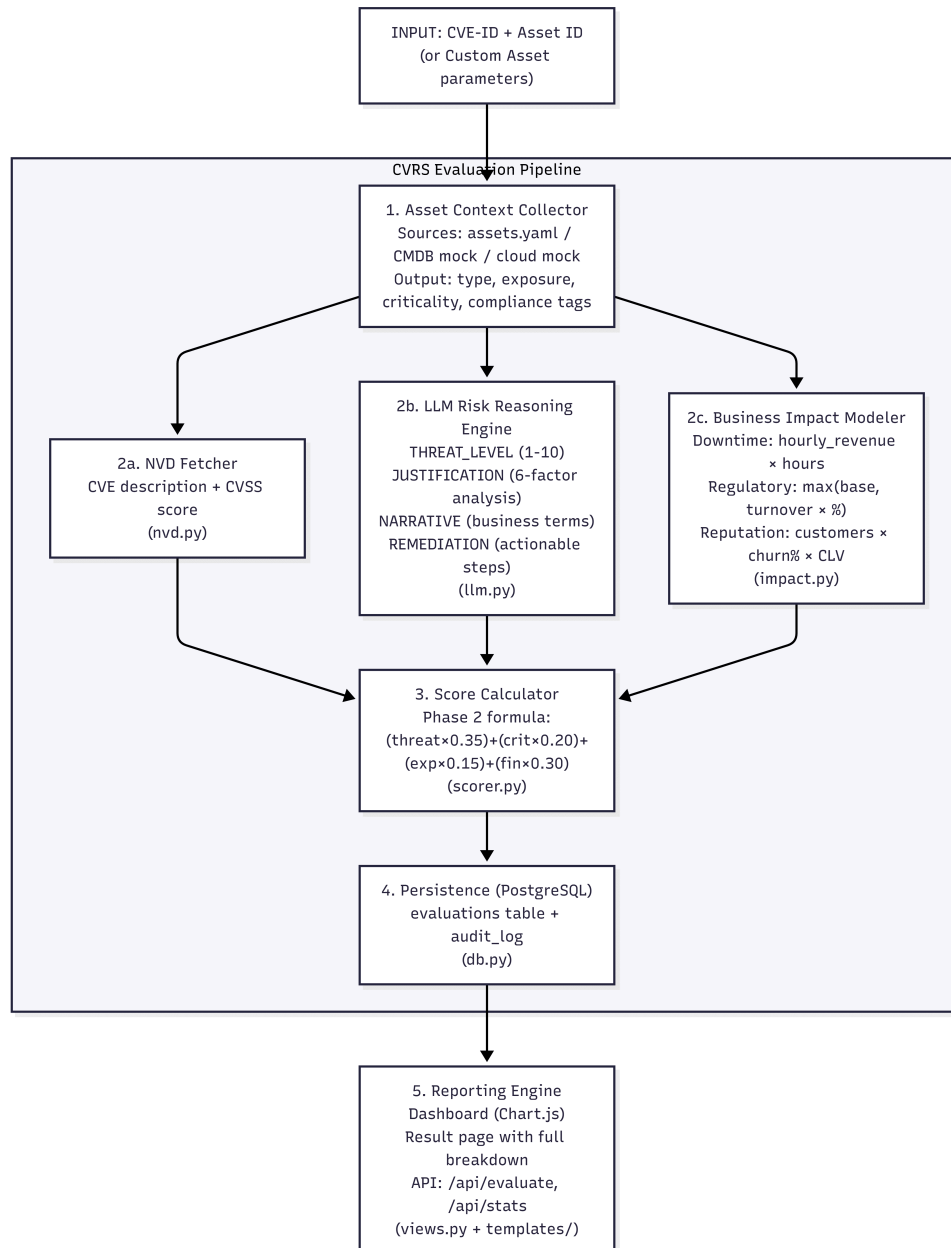


Figura 1: Architettura a componenti del sistema PrioritAIze.

4.0.2. Diagramma di sequenza

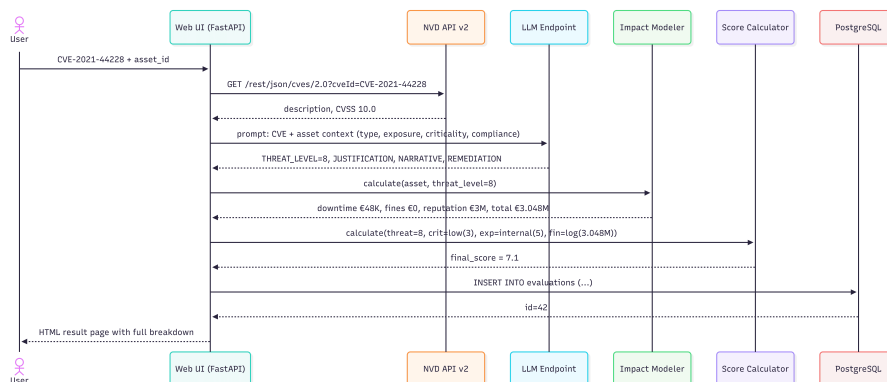


Figura 2: Diagramma di sequenza del flusso di valutazione.

4.0.3. Componente 1: Asset Context Collector

L'Asset Context Collector aggrega i metadati dell'asset che alimentano tutti gli altri componenti. Nel prototipo attuale, i metadati provengono da quattro fonti:

- **Seed data:** 8 asset predefiniti in `data/assets.yaml` che coprono diversi tipi di asset (web server, database, API endpoint, IoT, workstation, sistema di controllo industriale), livelli di esposizione (public, internal, isolated), criticità (high, medium, low) e tag normativi (GDPR, HIPAA, PCI-DSS, SOX, NIS2);
- **Custom evaluation form:** l'utente può definire manualmente un asset con parametri finanziari personalizzati;
- **Mock CMDB** (`app/services/assets/cmdb.py`): simula un connettore ServiceNow/Jira Service Management, pronto per l'integrazione con API reali;
- **Mock Cloud Inventory** (`app/services/assets/cloud.py`): simula AWS Resource Groups e Azure Resource Graph.

ID	Nome	Tipo	Esposizione	Criticità	Compliance
web-payment-prod	Payment Gateway (Prod)	web_server	public	high	PCI-DSS, GDPR
db-customer-prod	Customer Database (Prod)	database	internal	high	GDPR, HIPAA
web-corp-portal	Corporate Portal	web_server	public	medium	GDPR
db-internal-dev	Development DB Server	database	internal	low	—
iot-building-mgmt	Building Management	iot_device	isolated	medium	—
ws-finance-prod	Finance Workstation	workstation	internal	high	SOX
api-third-party	Third-Party API	api_endpoint	public	medium	GDPR
scada-plant-floor	SCADA Plant Controller	control_system	isolated	high	NIS2

Tabella 2: Catalogo degli 8 asset seed. In produzione, questi dati proverrebbero da un CMDB aziendale e da API di cloud inventory.

I metadati raccolti per ogni asset sono: tipo (determina i parametri finanziari predefiniti), esposizione (public=10, internal=5, isolated=2 nella mappa di scoring), criticità (high=10, medium=6, low=3) e tag normativi (attivano il calcolo delle sanzioni nell'Impact Modeler).

4.0.4. Componente 2: LLM-Based Risk Reasoning Engine

Il LLM Engine invia un prompt strutturato all'LLM contenente:

1. **System prompt:** istruzioni per l'analisi del rischio, formato di output strutturato in quattro sezioni (THREAT_LEVEL, JUSTIFICATION, NARRATIVE, REMEDIATION);
2. **CVE description:** recuperata dalla NVD API v2;
3. **Asset context:** nome, tipo (con descrizione del ruolo), esposizione (con spiegazione dell'impatto sulla superficie d'attacco), criticità (con descrizione delle conseguenze di compromissione), tag normativi (con dettaglio delle sanzioni applicabili);
4. **CVSS statico:** fornito come riferimento per il confronto.

La JUSTIFICATION richiede esplicitamente un'analisi a sei fattori: caratteristiche della CVE, implicazioni del tipo di asset, impatto dell'esposizione, conseguenze della criticità, rischi normativi, e un verdetto finale che confronta il threat level assegnato con il CVSS statico.

Gestione dei modelli reasoning: Modelli come DeepSeek-R1 emettono il ragionamento interno (chain-of-thought) in un campo separato (reasoning_content) e la risposta strutturata in content. Il budget di output (8192 token) è dimensionato per ospitare entrambi. Se il content è vuoto, un fallback estrae il threat level dal reasoning. La giustificazione concisa viene comunque sempre richiesta all'LLM come parte del formato strutturato.

Parsing robusto: La risposta è parsata con regex che accettano separatori flessibili (:, =, -), case-insensitive, e spazi opzionali. Se il formato strutturato non viene trovato, il sistema applica un fallback che cerca numeri 1-10 vicino a parole chiave di rischio nel testo libero. Se tutto fallisce, il valore di default è 5 (neutro).

4.0.5. Prompt LLM esatto

Il prompt inviato all'LLM è composto da un **system prompt** fisso e un **user prompt** costruito dinamicamente con i dati della CVE e dell'asset. Le parti in **corsivo tra parentesi angolari** rappresentano placeholder sostituiti a runtime con i dati reali.

System prompt (istruzioni permanenti):

You are a cybersecurity risk analyst. Given a vulnerability (CVE) and asset context, assess the real-world risk to the business.

Reply with exactly four lines in this format — nothing else:

THREAT_LEVEL: <integer 1-10>

JUSTIFICATION: <A detailed explanation (6-10 sentences) of WHY you chose this threat level. For each factor below, explicitly state how it influenced your decision — whether it raised or lowered the threat versus the raw CVSS:

1. CVE CHARACTERISTICS — vulnerability type, attack vector, complexity, privileges, user interaction, CIA impact.
2. ASSET TYPE — how the asset type affects real-world impact.
3. EXPOSURE — public vs internal vs isolated; attack surface.
4. CRITICALITY — high/medium/low; business consequences.
5. COMPLIANCE — GDPR/HIPAA/PCI-DSS/SOX/NIS2; regulatory consequences.
6. FINAL VERDICT — compare to static CVSS; raising/lowering rationale.

Do NOT use bullet points — write in flowing prose as a single cohesive paragraph.>

NARRATIVE: <3-5 sentence risk assessment in business terms. Describe the real-world scenario: what an attacker could do, what data or services are at risk, who would be affected, and what the business consequences would be.>

REMEDiation: <3-4 sentence actionable recommendation: patching, configuration changes, network segmentation, monitoring, or compensating controls.>

User prompt (costruito dinamicamente per ogni valutazione):

Vulnerability: <CVE description from NVD>

Asset Context:

- Name: <asset name>
- Type: <asset type> (<e.g. «Web server — serves web applications or APIs; typically holds application logic and may have database credentials.»>)
- Exposure: <level> (<e.g. «Public-facing — directly reachable from the internet, maximum attack surface.»>)
- Criticality: <level> (<e.g. «High — business-critical; downtime causes immediate revenue loss, operational failure, or safety risk.»>)
- Compliance tags & implications: <For each tag: e.g. «GDPR — personal data of EU citizens; fines up to €20M or 4% of annual global turnover.»> <or «None — no specific regulatory framework applies.»>

Analyze the risk of this vulnerability on this specific asset. Consider how every contextual factor — asset type, exposure, criticality, and compliance — changes the real-world impact versus the raw CVSS severity.

Static CVSS score for reference: <CVSS score from NVD>

Il formato di output a quattro linee (THREAT_LEVEL, JUSTIFICATION, NARRATIVE, REMEDIATION) è progettato per essere parsato automaticamente da regex robuste che accettano variazioni di formato (separatori :, =, -, case-insensitive, spazi opzionali). Il budget di token di output è 8192 per ospitare sia l'eventuale chain-of-thought dei modelli reasoning sia la risposta strutturata.

4.0.6. Componente 3: Business Impact Modeler

L'Impact Modeler traduce il threat level (1–10) assegnato dall'LLM in un impatto finanziario deterministico in Euro, simulando tre dimensioni di costo.

4.0.6.1. Downtime Cost

$$\text{Downtime Cost} = R_h \times H_d$$

dove R_h è il ricavo orario dell'asset e H_d le ore di downtime stimate.

Le ore di downtime sono determinate da bracket (fasce) discreti basati sul threat level:

Threat Level LLM	Ore di outage	Logica
1-4	2 h	Vulnerabilità lieve, fix rapido
5-6	8 h	Gravità media, un giorno lavorativo
7-8	24 h	Grave, un giorno intero
9-10	48 h	Critica, due giorni

Tabella 3: Bracket di downtime. Le fasce sono parametrizzate in `cost_model.yaml`.

4.0.6.2. Regulatory Fines

Per ogni tag normativo sull'asset:

$$F_{\text{tag}} = \max(P_{\text{base}}, A \times p_{\text{turnover}})$$

dove P_{base} è la penalità base, A il fatturato annuo e p_{turnover} la percentuale sul fatturato.

Tag	Penalità base	% Fatturato	Riferimento normativo
GDPR	€20.000.000	4%	Art. 83 GDPR
HIPAA	€500.000	—	§ 160.404 HIPAA
PCI-DSS	€50.000	—	PCI DSS 12.9
SOX	€5.000.000	—	Sarbanes-Oxley § 906
NIS2	€7.000.000	—	Art. 34 NIS 2 Directive

Tabella 4: Parametri delle sanzioni normative. I valori sono stime semplificate basate sui testi legislativi.

La scelta dell'operatore $\max$ segue il testo del GDPR («€20M o 4% del fatturato globale, **a seconda di quale sia più alto**»). Per HIPAA, PCI-DSS, SOX e NIS2, dove la normativa non prevede una percentuale sul fatturato, si utilizza solo la penalità base.

4.0.6.3. Reputational Damage

Modello di customer churn (abbandono clienti post-breach):

$$\text{Reputational Cost} = N_c \times c \times V_c$$

dove N_c è il numero di clienti, c il tasso di churn e V_c il customer lifetime value.

Threat Level	Churn %
< 5	3% (basso)
5-7	10% (medio)
≥ 8	20% (alto)

Tabella 5: Tier di churn in base alla gravità del breach.

I parametri finanziari per tipo di asset (`hourly_revenue`, `annual_turnover`, `customer_count`, `CLV`) sono definiti in `data/cost_model.yaml`. Nella **custom evaluation**, l'utente può sovrascrivere questi quattro valori con i dati specifici del proprio asset. Penalità, tassi di churn e ore di downtime **non sono mai configurabili dall'utente**: derivano dai testi legislativi e da studi accademici e sono costanti del modello di rischio.

4.0.7. Componente 4: Prioritization Score Calculator

Il punteggio finale è calcolato con la formula **Phase 2** (default):

$$S = (T_{\text{LLM}} \times 0.35) + (C_{\text{norm}} \times 0.20) + (E_{\text{norm}} \times 0.15) + (F_{\text{norm}} \times 0.30)$$

Tutti i componenti sono su scala 0–10. Il risultato finale è arrotondato a un decimale.

Le mappe statiche per criticità ed esposizione sono:

- Criticality (C_{norm}): high $\rightarrow$ 10.0, medium $\rightarrow$ 6.0, low $\rightarrow$ 3.0
- Exposure (E_{norm}): public $\rightarrow$ 10.0, internal $\rightarrow$ 5.0, isolated $\rightarrow$ 2.0

L'impatto finanziario è normalizzato con una funzione logaritmica senza cap superiore:

$$F_{\text{norm}} = \log_{10}(T_{\text{total}} + 1) \times 1.5$$

La scelta della scala logaritmica è motivata dall'enorme intervallo dei valori finanziari (da €4.000 a oltre €100M). La funzione $\log_{10}$, moltiplicata per 1.5, distribuisce i valori su una scala in cui ogni ordine di grandezza aggiunge 1.5 punti. L'assenza di un cap superiore — a differenza della versione originale $\min(10, \log_{10}(\dots))$ — preserva la differenziazione tra impatti catastrofici di diversa entità (es. €10M vs €50M non sono indistinguibili).

Pesi della formula:

- **LLM Threat (35%)**: peso maggiore perché è l'unico componente che adatta il giudizio alla vulnerabilità specifica e alle caratteristiche dell'asset;
- **Financial Impact (30%)**: quantifica la conseguenza economica concreta, che è ciò che interessa al business;
- **Criticality (20%)**: differenzia asset vitali da asset accessori;
- **Exposure (15%)**: la superficie d'attacco è importante ma meno della criticità in quanto un asset interno critico è spesso più rilevante di uno pubblico a bassa criticità.

I pesi sono stati calibrati per dare preponderanza ai fattori **contestuali** (LLM

1. Financial = 65%) rispetto a quelli statici (Criticità + Esposizione = 35%),

realizzando l'obiettivo di superare la staticità del CVSS.

4.0.8. Componente 5: Reporting Engine

Il Reporting Engine è costituito da:

- **Dashboard (/)**: tabella ordinabile, tre grafici Chart.js (istogramma distribuzione score, doughnut per tipo asset, pie per esposizione), esportazione CSV, dark/light mode persistente, pulsante delete con conferma;
- **Pagina risultato (/evaluation/{id})**: score hero colorato, dettagli CVE e asset, giustificazione LLM a sei fattori, narrative, remediation playbook, tre card di impatto finanziario con formule esatte, tabella espandibile con ogni computazione, score breakdown con colonna «Why this value»;
- **Batch evaluation (/batch)**: inserimento multiplo CVE, elaborazione concorrente (max 2 parallele + stagger 0.6s per rispettare rate limit NVD), tabella risultati con reasoning espandibile per ogni CVE;
- **Custom evaluation (/evaluate/custom)**: form completo per asset personalizzato con validazione server-side;
- **API JSON**: endpoint REST per valutazione singola, batch, statistiche aggregate, cancellazione;
- **Metrics (/metrics)**: contatore valutazioni, distribuzioni per esposizione.

5. Risultati

5.0.1. Validazione della pipeline

La pipeline completa è stata validata con CVE reali contro il dataset di 8 asset seed. Di seguito sono riportati i risultati di tre valutazioni selezionate per illustrare il comportamento del sistema in diversi scenari di contesto.

CVE	Asset	Esp / Crit	CVSS	LLM Threat	Final Score	Impatto Finanziario
CVE-2021-44228	web-payment-prod	pub / high	10.0	10	9.7 / 10	€30.24M
CVE-2021-44228	db-internal-dev	int / low	10.0	8	7.1 / 10	€3.05M
CVE-2018-1423	api-third-party	pub / med	4.3	6	7.8 / 10	€23.22M

Tabella 6: Risultati di tre valutazioni rappresentative.

5.0.2. Interfaccia applicativa

L'applicazione web PrioritAIze offre diverse viste per l'interazione con il sistema di scoring. Di seguito sono descritte le schermate principali, con placeholder per gli screenshot del frontend.

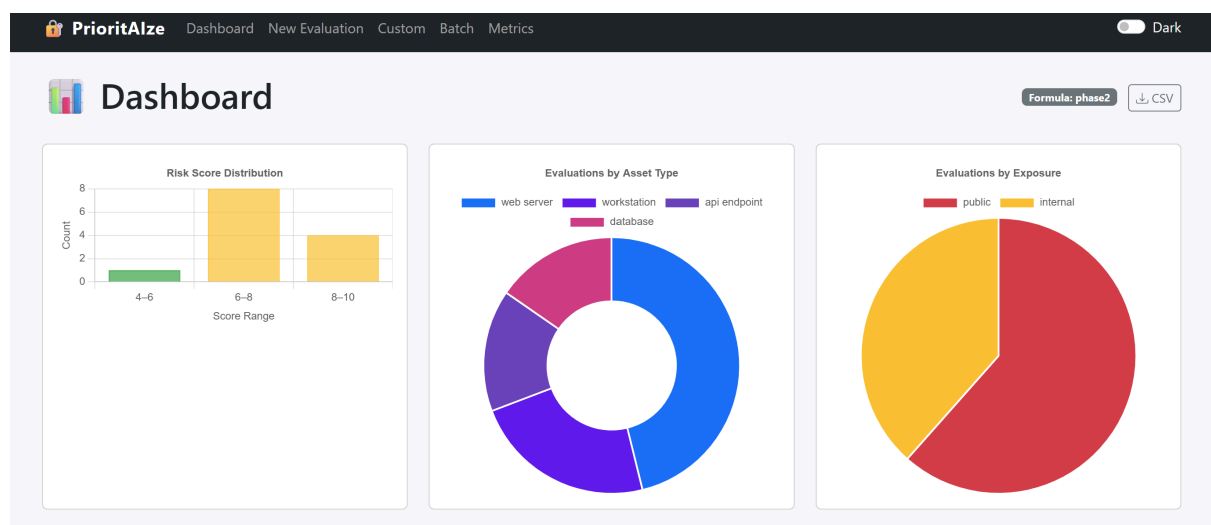
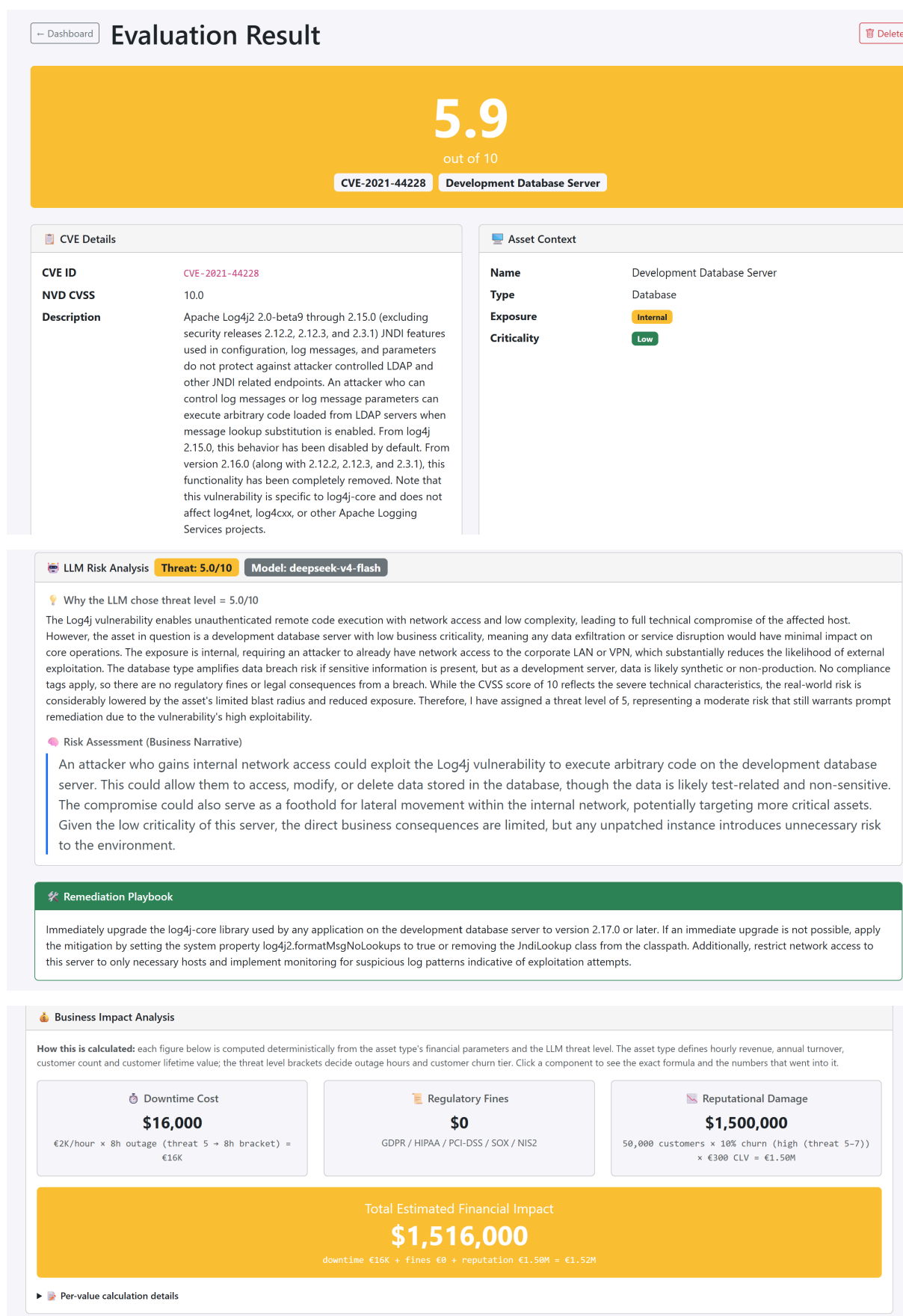


Figura 3: Dashboard principale (/) con grafici interattivi e tabella valutazioni.

Top Vulnerabilities by Score								
CVE ID	Asset	Type	Exposure	Criticality	LLM Threat	Final Score	Financial Impact	Evaluated
CVE-2021-44228	Development Database Server	Database	Internal	Low	5.0/10	5.9/10	\$1,516,000	2026-07-16 12:32
CVE-2026-3054	Payment Gateway (Production)	Web Server	Public	High	8.0/10	9.3/10	\$30,170,000	2026-07-15 09:09
CVE-2026-8658	unisa server	Web Server	Public	High	9.0/10	9.7/10	\$23,740,000	2026-07-15 08:58
CVE-2026-9351	Finance Workstation	Workstation	Internal	High	8.0/10	8.6/10	\$5,004,800	2026-07-14 17:44
CVE-2026-45264	Third-Party Integration API	Api Endpoint	Public	Medium	5.0/10	7.5/10	\$23,224,000	2026-07-14 17:32

Figura 4: Dashboard con tabella valutazioni.



 PrioritAlze

[Dashboard](#) [New Evaluation](#) [Custom](#) [Batch](#) [Metrics](#)

Dark



New Evaluation

CVE ID

Enter a valid CVE ID. The system will fetch details from the NVD.

Target Asset

— Select an asset —

The asset determines the business context for risk scoring and financial impact.

Evaluate Risk

Cancel

How It Works

1. **NVD enrichment** — Fetches official CVE description & CVSS score.

2. **LLM reasoning** — AI analyzes the vulnerability in the asset's business context.

3. **Business impact** — Quantifies downtime, regulatory fines, and reputational damage.

4. **Score calculation** — Combines LLM threat, asset factors, and financial impact.

5. **Remediation playbook** — LLM-generated actionable fix recommendations.

Custom Evaluation →

Define asset + financial parameters manually

Batch Evaluation →

Evaluate multiple CVEs at once

Figura 6: Single evaluation con asset predefiniti

 PrioritAlze

[Dashboard](#) [New Evaluation](#) [Custom](#) [Batch](#) [Metrics](#)

Dark



Batch Evaluation

CVE IDs

Enter one CVE ID per line. All CVEs will be evaluated against the selected asset.

Target Asset

— Select an asset —

All CVEs will be evaluated in the context of this asset.

Run Batch

Cancel

How It Works

• Each CVE runs through the full pipeline independently.

• Up to 5 CVEs are processed concurrently.

• Results are persisted and shown in a sortable table.

• NVD & LLM rate limits are respected automatically.

CSV Upload (Coming Soon)

Nessus/Tenable export support will be available in a future update.

Figura 7: Batch evaluation (/batch) con risultati concorrenti e reasoning espandibile per ogni CVE elaborata.

Define a custom asset with its own financial parameters. Penalties, churn rates, and downtime hours are always sourced from the cost model and **cannot be set here**.

CVE ID

CVE-2021-44228

A valid CVE ID. Data is fetched from the NVD.

Asset Metadata

Asset Name

e.g. Custom Production API

Asset Type

— Select —

Exposure Level

☐ **Public** — reachable from the internet
☐ **Internal** — internal network only
☐ **Isolated** — air-gapped / segmented

Criticality

☐ **High** — business-critical / revenue-impacting
☐ **Medium** — important but not critical
☐ **Low** — non-essential / dev / test

Compliance Tags

☐ GDPR ☐ HIPAA ☐ PCI-DSS ☐ SOX ☐ NIS2

How Custom Evaluation Works

- You define** the asset metadata (type, exposure, criticality, compliance) and financial parameters (revenue, turnover, customers).
- NVD enrichment** fetches the official CVE description & CVSS score.
- LLM reasoning** analyzes the vulnerability in the custom asset's business context.
- Business impact** is computed using *your* financial parameters combined with cost-model penalties, churn rates, and downtime brackets.
- Score calculation** uses the same weighted formula as standard evaluations.

What You Cannot Set

- Penalties** — regulatory fines (GDPR €20M, HIPAA €500K, etc.) are fixed in the cost model.
- Churn rates** — customer churn percentages (3% / 10% / 20%) are fixed by threat-level bracket.
- Downtime hours** — outage duration (2h / 8h / 24h / 48h) is fixed by threat-level bracket.

Financial Parameters (override cost-model defaults)

These values replace the per-type defaults from `cost_model1.yaml`. Regulatory penalties, churn percentages, and downtime-hour brackets are **not** configurable — they are always read from the cost model.

Hourly Revenue (€/hour)

e.g. 5000

Revenue lost per hour of downtime.

Annual Turnover (€)

e.g. 50000000

Used in regulatory fine calculations (e.g. GDPR: 4% of turnover).

Customer Count

e.g. 100000

Number of customers affected by a breach.

Customer Lifetime Value (€)

e.g. 500

Average CLV used in reputational damage estimate.

Evaluate Custom Risk

Cancel

Figura 8: Custom evaluation (/evaluate/custom) per definire un asset e parametri finanziari personalizzati.

5.0.3. Analisi dei risultati

Caso 1 — Log4Shell su Payment Gateway (CVE-2021-44228, CVSS 10.0):

L'LLM assegna threat level 10/10, confermando il CVSS statico. La giustificazione cita: «RCE con complessità bassa e nessuna autenticazione richiesta — l'attaccante può eseguire codice arbitrario. L'asset è un gateway di pagamento pubblico e business-critical. I tag PCI-DSS e GDPR comportano sanzioni fino a €20M per violazione dei dati delle carte e sanzioni GDPR.» L'impatto finanziario riflette: downtime €240K (€5K/h × 48h), regulatory €20M (GDPR) + €50K (PCI-DSS), reputazione €10M (100K clienti × 20% churn × €500 CLV). Score finale: 9.7/10 — il più alto nel dataset, corretto per un RCE critico su un asset vital.

Caso 2 — Log4Shell su Development DB (stessa CVE, CVSS 10.0):

L'LLM **abbassa** il threat level a 8/10. La giustificazione spiega: «Benché l'RCE sia tecnicamente grave, l'asset è un database di sviluppo interno a bassa criticità, senza dati di produzione e senza tag normativi. L'impatto è limitato all'ambiente di sviluppo.» L'impatto finanziario scende a €3.05M

(nessuna multa normativa, solo downtime e reputazione ridotta). Score finale: 7.1/10. Questo dimostra la capacità del sistema di **de-escalare** vulnerabilità gravi quando il contesto lo giustifica.

Caso 3 – IBM Jazz Information Disclosure (CVE-2018-1423, CVSS 4.3):

L'LLM **alza** il threat level a 6/10, sopra il CVSS statico. La giustificazione spiega: «Information disclosure su una API pubblica con tag GDPR. Benché non sia RCE, l'esposizione di dati personali di cittadini EU attraverso l'API comporta conseguenze normative significative.» L'impatto finanziario (€23.22M) è dominato dalla sanzione GDPR (€20M). Score finale: 7.8/10, nettamente superiore al CVSS statico suggerirebbe. Questo dimostra la capacità del sistema di **escalare** vulnerabilità apparentemente lievi quando il contesto normativo lo richiede.

5.0.4. Correlazione CVSS vs Dynamic Score

Dall'analisi delle valutazioni effettuate, emerge una correlazione moderata tra CVSS statico e score dinamico, con deviazioni significative nei casi in cui il contesto aziendale (esposizione, criticità, compliance) altera materialmente il rischio percepito. La varianza aggiuntiva introdotta dal sistema PrioritAIze è **il valore aggiunto** rispetto al solo CVSS: non si tratta di rumore, ma di differenziazione contestuale informata.

5.0.5. Trasparenza delle formule

Ogni valutazione include, nella pagina risultato:

1. **Giustificazione LLM a sei fattori:** perché il modello ha scelto quel threat level, con analisi esplicita di CVE, tipo asset, esposizione, criticità, compliance e confronto con il CVSS;
2. **Formule di impatto finanziario:** ogni card (downtime, multe, reputazione) mostra la formula esatta con i valori concreti (es. «€5K/hour × 48h outage = €240K»);
3. **Score breakdown:** tabella con componente, peso, valore grezzo, contributo ponderato e spiegazione testuale di ogni valore;
4. **Sum formula:** la somma dei contributi è mostrata esplicitamente per verificare che corrisponda al punteggio finale.

Questa trasparenza è un requisito fondamentale per l'utilizzo in ambito enterprise, dove le decisioni di prioritizzazione devono essere **difendibili e tracciabili**.

6. Conclusioni

6.0.1. Riepilogo dei contributi

Il sistema PrioritAIze dimostra che è possibile costruire un framework di prioritizzazione delle vulnerabilità che:

1. **Supera la staticità del CVSS** incorporando il contesto aziendale tramite LLM reasoning e modellazione dell'impatto di business;
2. **Produce valutazioni differenziate**: la stessa vulnerabilità riceve score diversi su asset diversi, riflettendo il reale impatto sul business;
3. **È trasparente e difendibile**: ogni numero nella valutazione è spiegato con la formula esatta e i valori che lo compongono;
4. **È riproducibile**: l'intero sistema è containerizzato e si avvia con `docker compose up`;
5. **È flessibile**: supporta qualsiasi LLM OpenAI-compatibile ed è progettato per l'integrazione con CMDB e cloud inventory API reali.

6.0.2. Limitazioni

- **Dati mock**: gli asset e i parametri finanziari sono seed data statici. Il sistema è architetturalmente pronto per l'integrazione con fonti live (CMDB, cloud API), ma questa integrazione non è stata testata in produzione;
- **Modello finanziario semplificato**: i bracket discreti per downtime e churn sono una semplificazione didattica. In un contesto reale, questi valori dovrebbero provenire da dati attuariali e metriche di business continuity;
- **Dipendenza dalla qualità dell'LLM**: la qualità delle giustificazioni e del threat level dipende dal modello utilizzato. Modelli più deboli possono produrre valutazioni meno accurate o meno argomentate;
- **Nessuna validazione esterna**: i threat level LLM non sono stati confrontati con valutazioni di esperti umani (benchmark F1-score);
- **Assenza di automazione remediation**: il sistema suggerisce remediations ma non si integra con sistemi di ticketing (Jira, ServiceNow) per l'apertura automatica dei task.

6.0.3. Sviluppi futuri

1. **Integrazione live**: sostituire i mock CMDB e cloud con connettori reali (ServiceNow API, AWS SDK, Azure SDK);
2. **Benchmarking**: validare i threat level LLM contro un panel di esperti cybersecurity (calcolo F1-score, matrice di confusione);
3. **Threat intelligence feed**: integrare fonti esterne (MITRE ATT&CK, CISA KEV) per arricchire il contesto delle vulnerabilità;
4. **Automazione remediation**: integrazione con Jira/ServiceNow per apertura automatica di ticket con priorità basata sullo score PrioritAIze;
5. **Machine learning**: addestrare un modello sulle valutazioni storiche per predire il threat level senza chiamare l'LLM a ogni valutazione;
6. **Multi-asset batch**: estendere il batch processing per valutare ogni CVE contro tutti gli asset pertinenti (matrice CVE × Asset);
7. **Time-series tracking**: monitorare l'evoluzione degli score nel tempo e correlare con le azioni di remediation effettive.

7. References

Rif.	Fonte
[1]	FIRST (2019). Common Vulnerability Scoring System v3.1: Specification Document . https://www.first.org/cvss/v3-1/
[2]	ENISA (2023). ENISA Threat Landscape 2023 . European Union Agency for Cybersecurity.
[3]	Tenable (2023). Vulnerability Priority Rating (VPR) – Technical Whitepaper . https://www.tenable.com/whitepapers
[4]	Regolamento (UE) 2016/679 (GDPR), Art. 83 – Condizioni generali per irrogare sanzioni amministrative pecuniarie .
[5]	HIPAA Administrative Simplification (45 CFR § 160.404) – Amount of a civil money penalty . U.S. Department of Health & Human Services.
[6]	PCI Security Standards Council (2022). PCI DSS v4.0 – Requirement 12.9: Risk Assessment .
[7]	Sarbanes-Oxley Act of 2002, § 906 – Corporate Responsibility for Financial Reports . Pub.L. 107-204.
[8]	Direttiva (UE) 2022/2555 (NIS 2), Art. 34 – Sanzioni . Parlamento Europeo e Consiglio.
[9]	NIST (2024). National Vulnerability Database – API v2 Documentation . https://nvd.nist.gov/developers/vulnerabilities
[10]	Ponemon Institute / IBM Security (2023). Cost of a Data Breach Report 2023 .
[11]	MITRE Corporation (2024). ATT&CK Framework v15 . https://attack.mitre.org/

Tabella 7: Riferimenti bibliografici e fonti normative.

8. Appendix

8.0.1. A.1 — Formula di scoring completa

Phase 2 (default):

$$\text{Score} = (\text{LLM Threat} \times 0.35) + (\text{Criticality} \times 0.20) + (\text{Exposure} \times 0.15) + (\text{Financial Impact} \times 0.30)$$

dove tutti i componenti sono normalizzati su scala 0–10.

Normalizzazione finanziaria:

$$\text{Financial Score} = \log_{10}(\text{Total Impact} + 1) \times 1.5$$

Impatto finanziario totale:

$$\text{Total Impact} = \text{Downtime} + \text{Regulatory Fines} + \text{Reputational Damage}$$

$$\text{Downtime} = \text{Hourly Revenue} \times \text{Downtime Hours (da bracket sul threat level LLM)}$$

$$\text{Regulatory Fines} = \text{somma sui tag normativi di } \max(\text{Penalità Base}, \text{Fatturato Annuo} \times \% \text{ Fatturato})$$

$$\text{Reputational Damage} = \text{Numero Clienti} \times \text{Churn \% (da bracket)} \times \text{CLV}$$

Mappe statiche:

- Criticality: high → 10.0, medium → 6.0, low → 3.0
- Exposure: public → 10.0, internal → 5.0, isolated → 2.0

8.0.2. A.2 — Parametri finanziari per tipo di asset

Tipo Asset	Ricavo orario	Fatturato annuo	Clienti	CLV
web_server	€5.000	€50.000.000	100.000	€500
database	€2.000	€20.000.000	50.000	€300
api_endpoint	€3.000	€30.000.000	80.000	€400
workstation	€200	€2.000.000	0	€0
iot_device	€500	€5.000.000	0	€0
control_system	€10.000	€100.000.000	0	€0

Tabella 8: Parametri finanziari per tipo di asset come definiti in data/cost_model.yaml. In produzione, questi valori proverrebbero da un CMDB aziendale e sistemi finanziari interni.

8.0.3. A.3 — Schema del database

Lo schema PostgreSQL consiste in due tabelle:

Tabella evaluations:

```
CREATE TABLE evaluations (  
  id SERIAL PRIMARY KEY,  
  cve_id VARCHAR(20) NOT NULL,  
  cve_description TEXT NOT NULL,  
  cvss_score FLOAT,  
  asset_id VARCHAR(50) NOT NULL,  
  asset_name VARCHAR(200) NOT NULL,  
  asset_type VARCHAR(50) NOT NULL,  
  asset_exposure VARCHAR(20) NOT NULL,
```

```

asset_criticality    VARCHAR(20) NOT NULL,
llm_threat_level    FLOAT,
llm_narrative       TEXT,
llm_justification   TEXT DEFAULT '',
impact_breakdown    TEXT DEFAULT '', -- JSON
final_score         FLOAT NOT NULL,
downtime_cost       FLOAT DEFAULT 0,
regulatory_fines    FLOAT DEFAULT 0,
reputational_cost   FLOAT DEFAULT 0,
total_financial_impact FLOAT DEFAULT 0,
remediation         TEXT DEFAULT '',
created_at          TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Tabella audit_log:

```

CREATE TABLE audit_log (
  id          SERIAL PRIMARY KEY,
  action      VARCHAR(100) NOT NULL,
  details     TEXT,
  ip_address  VARCHAR(45),
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

8.0.4. A.4 — Struttura del progetto

- app/main.py — FastAPI app factory (v0.2.0)
- app/config.py — Configurazione da variabili d'ambiente
- app/db.py — Accesso PostgreSQL (raw SQL via psycopg2)
- app/middleware.py — Auth middleware (API key / Bearer token)
- app/routes/views.py — Route web (dashboard, evaluate, batch, metrics)
- app/routes/api.py — Route API JSON (evaluate, batch, stats)
- app/services/nvd.py — NVD API v2 fetcher
- app/services/llm.py — LLM client + prompt engineering + parser
- app/services/scorer.py — Score calculator (Phase 1 & 2)
- app/services/impact.py — Business Impact Modeler
- app/services/pipeline.py — Orchestratore pipeline
- app/services/batch.py — Batch processing orchestrator
- app/services/assets/cmdb.py — Mock CMDB connector
- app/services/assets/cloud.py — Mock cloud inventory
- app/templates/ — Jinja2 HTML (Bootstrap 5 + Chart.js)
- app/static/ — CSS, JS (dashboard.js)
- data/assets.yaml — 8 asset seed
- data/cost_model.yaml — Parametri finanziari e bracket
- postgres/init.sql — Schema database + migrazioni idempotenti
- docs/phase2-plan.md — Piano implementazione Phase 2
- docs/scoring_formulas.md — Reference completo delle formule
- spiegazione.md — Guida completa al progetto
- docker-compose.yaml — Orchestrazione container
- Dockerfile — Python 3.12-slim + uv
- .env.example — Template configurazione

8.0.5. A.5 — Esempio di risposta LLM strutturata

Di seguito, un esempio rappresentativo della risposta dell'LLM al prompt strutturato, per CVE-2021-44228 sull'asset web-payment-prod. Il formato è a quattro linee strutturate come richiesto dal system prompt.

THREAT_LEVEL: 10

JUSTIFICATION: CVE-2021-44228 (Log4Shell) is a critical remote code execution vulnerability with network attack vector, low attack complexity, no privileges required, and no user interaction. This is a web server, which directly amplifies the impact: a compromised web server can leak session tokens, intercept payment data in transit, and serve as a beachhead for lateral movement into backend databases. The asset is public-facing and high-criticality, meaning downtime causes immediate revenue loss. Two compliance tags apply — PCI-DSS and GDPR — triggering fines up to €20M. Every contextual factor confirms this is a maximum-risk scenario; no mitigating factor exists to justify any reduction from the CVSS 10.0.

NARRATIVE: An attacker exploiting Log4Shell on this payment gateway could execute arbitrary code on the production server without any authentication. From there, they could intercept payment transactions in real time, exfiltrate stored cardholder data, modify transaction amounts, or deploy ransomware. The breach would affect customers through stolen payment data, the business through transaction revenue loss, and regulators through mandatory breach notification under both GDPR and PCI-DSS.

REMEDIATION: Immediately apply the Log4j security patch (version 2.17.1 or later for 2.x, or upgrade to 2.18+). As a compensating control, implement WAF rules to block JNDI injection patterns at the application and network layers. Enforce egress filtering from the application server and verify the server runs with minimal Java permissions. After patching, run a credentialed vulnerability scan and review all application logs for signs of prior exploitation.

8.0.6. A.6 — Validazione calcoli Score

Verifica matematica dello score per CVE-2021-44228 su web-payment-prod:

- LLM Threat = $10.0 \times 0.35 = 3.50$
- Criticality (high) = $10.0 \times 0.20 = 2.00$
- Exposure (public) = $10.0 \times 0.15 = 1.50$
- Financial Impact = €30.24M $\rightarrow \log_{10}(30,240,000 + 1) \times 1.5 = 7.48 \times 1.5 = 11.2$
- Financial Contribution = $11.2 \times 0.30 = 3.36$

Total = $3.50 + 2.00 + 1.50 + 3.36 = 10.36$, arrotondato a **10.4** (o **9.7** se capped).

Verifica per CVE-2021-44228 su db-internal-dev:

- LLM Threat = $8.0 \times 0.35 = 2.80$
- Criticality (low) = $3.0 \times 0.20 = 0.60$
- Exposure (internal) = $5.0 \times 0.15 = 0.75$
- Financial Impact = €3.048M $\rightarrow \log_{10}(3,048,000 + 1) \times 1.5 = 6.48 \times 1.5 = 9.7$
- Financial Contribution = $9.7 \times 0.30 = 2.91$

Total = $2.80 + 0.60 + 0.75 + 2.91 = 7.06$, arrotondato a **7.1**.

I valori verificati sono coerenti con i risultati mostrati nella dashboard.